

How the Computer Finds Branches

A walk through the whole pipeline — explained simply, and honestly — Branchseed Navigator,
`detector/branchseed.py`

Imagine the big blood vessel called the **aorta** is a highway running through the body, and the smaller blood vessels branching off it are **side streets**. The computer's job is to look at a 3D photo of the inside of the body (a CT scan) and figure out: where do side streets connect to the highway, which way do they go, and how wide are they? Here is exactly how it does that, step by step, in the order the code actually does it.

1 The glowing photo and the highlighter

A CT scan isn't a normal photo — it's more like a 3D X-ray where brighter spots usually mean denser stuff, and the blood vessels (filled with contrast dye) glow extra bright. Along with the scan, we're given a **mask**: a highlighter that's already colored in exactly where the highway (the aorta) is. The computer doesn't have to find the highway itself — that part is given to it.

2 Zoom in

Looking at the entire body would be slow and pointless — branches only come off the highway, not from far away. So the computer draws a box around the highlighted highway with a little extra room around it (25 mm, about the width of two fingers) and only looks inside that box from now on.

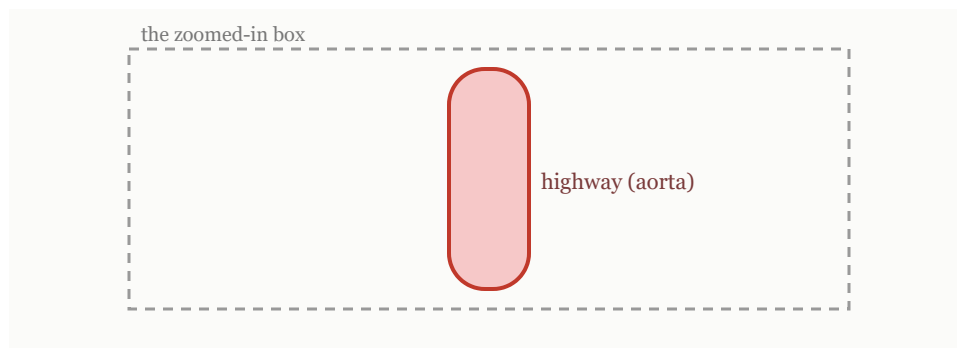


Fig. 1 — Only the boxed-in area around the highlighted highway is examined.

3 Decide what counts as "bright enough"

Every scan is a little different — some are brighter overall, some dimmer. So instead of using one fixed brightness number for every scan, the computer looks at how bright the *inside of the highway itself* is, and how bright the area *just around it* is, and picks a brightness cutoff in between, tuned specifically to this scan. Anything at least that bright is allowed to be considered "maybe part of a branch."

Why not just use one fixed number for every scan? Because scans from different machines or different patients can be brighter or dimmer overall. A brightness level that means "blood vessel" on one scan might mean "background" on another. Measuring each scan's own contrast first keeps the cutoff fair.

4 Only search in a ring around the highway, not at its very tips

The computer draws an imaginary ring around the highway — out to about 15 mm away — and only looks for branches inside that ring. It also skips the very ends of the cropped highway segment (about 5 mm from each tip), because a branch popping out of the very edge of the picture is more likely to be an artifact of where we happened to cut the box than a real side street.

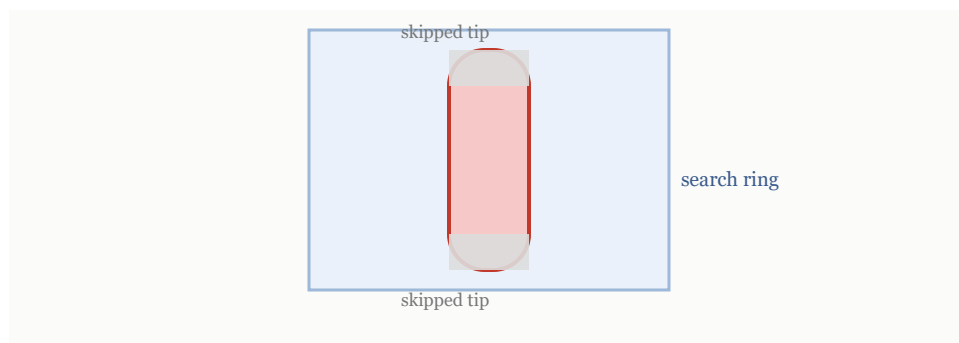


Fig. 2 — The blue ring is where the computer looks; the gray caps at each end are skipped.

5 Find the bright blobs

Inside that ring, the computer marks every spot that's bright enough (from Step 3), then glues together any bright spots that are touching each other into "blobs." Each blob is a **candidate branch** — something that might be a side street, but hasn't been confirmed yet.

6 For each blob, work out four things

Now the computer looks at each blob one at a time and tries to answer four questions about it.

a) Where's the doorway? (the ostium)

It finds every spot on the blob that's actually touching the highway wall, and picks the middle one. That's the "doorway" — where the side street connects to the highway.

b) Which way does it point? (the direction)

It looks at the shape of the blob *near the doorway* (within about 10 mm) and draws the single best straight arrow through those nearby dots — like laying a ruler over a handful of dots and finding the line that fits them best.

Why only the nearby dots, and not the whole blob? If a side street curves, using *every* dot in the whole blob (including far-away, curved parts) to draw one straight arrow gives you an arrow that cuts across the curve instead of pointing the way it actually starts out. Using only the nearby dots keeps the arrow honest about which way the street heads *right at the doorway*, which is the part that actually matters.

c) Where does the branch "start"? (the seed)

The computer doesn't hunt for this — it just walks 5 mm from the doorway, along the arrow from (b), and drops a pin there. That pin is called the "seed."

d) How wide is it? (the radius)

It pretends the nearby part of the branch is shaped like a smooth soda can, counts how many bright dots are packed into that space, and does some math to work out how fat a can would have to be to hold that many dots. That's the width guess.

Bonus: how sure is the computer? (confidence)

It gives itself a score between 0 and 1, mixing two things: "does this blob keep going for a

good while without stopping?" and "is it noticeably brighter than the background?" More of both = a higher confidence score.

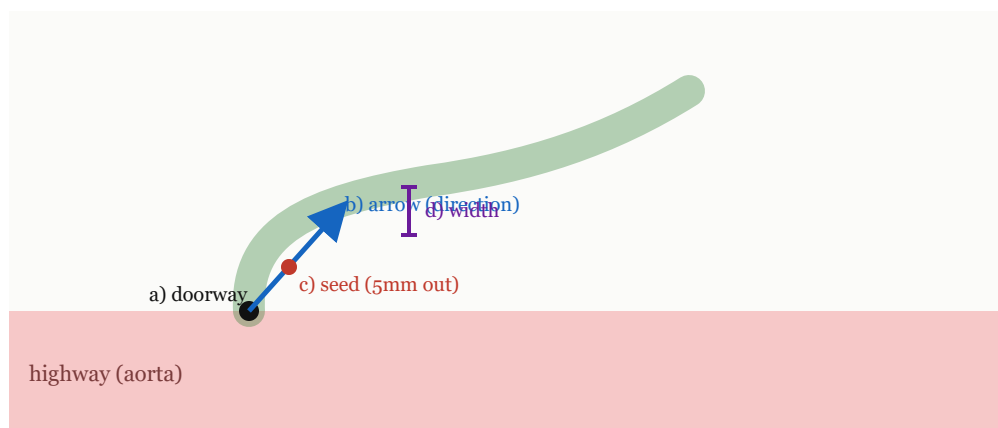


Fig. 3 — For every blob: doorway, arrow, a 5mm seed pin, and an estimated width.

7 Throw out duplicates

Sometimes the computer finds what looks like two separate blobs for the same real branch (maybe it got split into two pieces by a bright/dim gap). So it checks: do any two candidate branches have doorways very close together *and* point roughly the same way? If so, it keeps only the one it's more confident about and throws the other one away.

8 Package it up and double-check

The surviving branches get sorted and named `branch_001`, `branch_002`, and so on. Before anything is saved, the computer runs one last check on its own homework — making sure every branch has a real doorway, a direction arrow that's exactly length 1 (a proper compass direction), and a positive width. If anything looks broken, it refuses to save bad output.

The Honest Part

This entire process is **geometry and brightness math** — it doesn't know what a real artery looks like, doesn't know anatomy, and isn't a doctor. It's a fast, blunt guesser that works well when a branch is short, straight, and clearly separated from its neighbors. It can be fooled when:

- A branch **curves** a lot right after the doorway (partly why the direction arrow now only looks at nearby dots — to reduce, not eliminate, this problem).
- Two branches are **touching or fused together** in the picture — the computer may blend them into one confused blob.
- A branch is **very short** — there isn't much shape to measure, so the doorway, arrow, and width guesses are all shakier.
- The scan's brightness pattern is **unusual** — the "bright enough" cutoff from Step 3 might include some noise or miss a genuinely faint branch.

None of this makes it useless — it means the output should be treated as a good first guess to be checked, not a guaranteed truth.